PA3
AGENTIC
PROTOTYPE

Author

MAGGIE MICHELSEN
Utah State Univeristy

2026
ECE 5930: AI ARCHITECTURE
https://github.com/TheGodAmongstMen/AI_Architecture/tree/main#

In this Report

1 System Overview **2**

2 Agent Design **2**
2.1 GFWClient (Data Retrieval Agent) . . . 2
2.2 GeoFilter (Geospatial Processing Agent) . . . 2
2.3 LLMAgent (Explanation Agent) . . . 3
2.4 FishingAgent (Orchestrator Agent) . . . 3

3 Reliability and Security Decisions **3**
3.1 Retry Strategy . . . 3
3.2 Circuit breakers . . . 3
3.3 Idempotency . . . 4
3.4 Trust boundaries . . . 4
3.5 Memory security . . . 4

4 Deployment Plan **4**
4.1 Container Strategy . . . 4
4.2 Secrets Management . . . 4
4.3 State Management . . . 5
4.4 Edge Consideration . . . 5

5 Failure Mode Analysis **5**

6 Failure Injection Report **6**
6.1 Failure Injection One: . . . 6
6.2 Failure Injection Two: . . . 6

7 Reflection **7**
7.1 The PA2 Comparison . . . 7
7.2 My Most Fragile Component . . . 7
7.3 The Model Replacement Test . . . 7
7.4 What I Would Build Differently . . . 7

A requirements.txt **8**

1 System Overview

This system is designed to identify and explain potentially suspicious fishing activity using satellite-based data. It combines data collection, geospatial processing, machine learning, and basic language generation into one workflow that can be run automatically over a selected time period.

The process starts with the GFWClient, which pulls fishing activity data from the Global Fishing Watch API for a given date range and region. This data comes in a nested JSON format, so the GeoFilter class is used to clean and reorganize it into a GeoDataFrame. Each record is converted into a geographic point with associated information like time, vessel identifiers, and fishing effort.

Once the data is structured, the VesselAnalyzer handles the main analysis. It first applies DBSCAN clustering to group together nearby activity points, which can reveal areas where vessels are repeatedly operating. Then it calculates how long each fishing event lasts and uses an Isolation Forest model to detect anomalies. These anomalies represent vessels whose behavior differs from typical patterns, such as unusually long fishing durations or irregular activity levels.

After identifying these outliers, the system passes them to the LLMAgent, which generates short explanations describing why the vessels might be considered suspicious. Instead of just outputting raw data, this step helps make the results easier to understand by summarizing patterns like clustering, repeated presence, or unusual behavior.

The overall workflow is managed by the FishingAgent, which connects all the components and keeps a record of past runs. In the end, the system not only flags unusual fishing activity but also provides simple explanations, making it more useful for analysis or monitoring purposes.

2 Agent Design

The system is broken up into a few small agents, each with a specific job. This makes the pipeline easier to understand and avoids mixing responsibilities like data collection, analysis, and explanation.

2.1 GFWClient (Data Retrieval Agent)

This agent is responsible for pulling fishing activity data from the Global Fishing Watch API. It takes a start date, end date, and region ID as input, and returns raw JSON data. It's only allowed to make API requests and doesn't do any processing on the data itself. This keeps it simple and predictable. It doesn't keep any context between runs (each request is independent). Confidence is basically tied to whether the API call succeeds—if it fails, it raises an error. The output passed forward is just the raw JSON data, without extra request details like headers or debug info.

2.2 GeoFilter (Geospatial Processing Agent)

The GeoFilter takes the raw JSON and converts it into a clean GeoDataFrame. It extracts useful fields and turns latitude/longitude into geometry points. Its job is just formatting and organizing data, so it doesn't do any analysis or modeling. It uses geospatial tools like GeoPandas but avoids anything related to machine learning. There's no real context tracking here either—it just processes whatever it's given. If the dataset ends up empty or missing coordinates, that's effectively a low-confidence result. The output is a clean, structured dataset, and it intentionally drops the original nested JSON to keep things simpler downstream.

VesselAnalyzer (Analysis Agent) This agent handles the actual analysis. It takes the GeoDataFrame and adds cluster labels (using DBSCAN) and anomaly flags (using Isolation Forest). It's allowed to use these ML tools, but it doesn't call APIs or generate text. The idea is to keep it focused only on numeric and statistical analysis. It also doesn't store past results—everything is based on the current dataset. Low confidence here would look like weak or meaningless results, like everything ending up in one cluster or no anomalies being detected at all. The output includes the updated dataset with cluster and anomaly columns, but not the internal model details.

2.3 LLMAgent (Explanation Agent)

The LLMAgent takes the anomalies and turns them into a short explanation. Its input is a small sample of suspicious records, and its output is plain text. It's allowed to use a local language model, but it can't access external data or change the dataset. To manage context, it limits how many records go into the prompt (for example, taking the first 15 rows). If there's too much data, it just samples instead of trying to include everything. Low-confidence outputs here are usually vague or generic explanations that don't really refer to the data. The output is just the explanation text, without including the raw prompt or full dataset.

2.4 FishingAgent (Orchestrator Agent)

The FishingAgent ties everything together. It runs the full pipeline from data collection to explanation and returns both the processed dataset and the final summary. It can call all the other agents, but it doesn't do any heavy processing itself. It also keeps a simple history of past runs, although right now that's mostly for tracking rather than decision-making. If the history gets too long, it can be compressed into summaries. Overall confidence depends on the earlier steps—if no anomalies are found or the explanation is weak, the final result is less reliable. The final output includes only the analyzed data and explanation, leaving out intermediate steps like raw API responses.

3 Reliability and Security Decisions

3.1 Retry Strategy

The system retries transient failures such as API timeouts or temporary network errors when calling the Global Fishing Watch API. A simple exponential backoff strategy is used, where each retry waits progressively longer (e.g., 1s, 2s, 4s). Non-retriable errors, such as invalid parameters or authentication failures, are surfaced immediately without retrying.

3.2 Circuit breakers

Circuit breakers are primarily applied to the external API dependency (Global Fishing Watch). If repeated failures occur (e.g., 3–5 consecutive failed requests), the system temporarily stops making new requests to avoid overloading the API or wasting resources. During this "open" state, the pipeline can either return a failure message or fall back to previously cached results if available. After a cooldown period, the system allows a limited number of test requests to determine if the API has recovered.

3.3 Idempotency

Most operations in this system are read-only, especially API calls and data processing steps, so idempotency is naturally maintained. The only stateful component is the FishingAgent history log, which appends summaries of past runs. To avoid duplicate entries (for example, if a run is retried), each run can be uniquely identified by its input parameters (start date, end date, region). Before writing to history, the system can check whether an identical entry already exists. This ensures that repeated executions with the same inputs do not create duplicate records.

3.4 Trust boundaries

Externally retrieved data (from the API) is treated as untrusted input. It is never directly inserted into system prompts or instructions without filtering. The GeoFilter acts as a boundary by extracting only specific fields and discarding unexpected or malformed data. When passing data to the LLM, only a controlled subset of fields is included, and it is embedded as plain data rather than instructions. This reduces the risk of prompt injection or unintended influence on the model's behavior.

3.5 Memory security

The system's persistent memory (the history field in the FishingAgent) is limited and structured. Write-time validation ensures that only properly formatted summaries are stored, including checks for valid date ranges and non-empty explanations. Since the stored data is generated internally (not directly from user or API input), the risk of malicious content is reduced. Additionally, no raw external data is stored in memory—only summarized outputs—further limiting exposure to potentially unsafe inputs.

4 Deployment Plan

4.1 Container Strategy

All external Python dependencies are listed in Appendix A (requirements.txt). The system additionally depends on the Global Fishing Watch API. Each part of the system is packaged as a separate Python module with a single, clearly defined responsibility. This keeps the code organized and makes it easier to test or replace individual components without affecting the rest of the pipeline.

Overall, the system follows a modular, file-based structure rather than a monolithic design. Each component exposes a small interface (usually one or two main methods), which keeps dependencies between modules minimal and makes the system easier to extend or debug.

4.2 Secrets Management

Credentials are stored in a config.py file and are injected in gfw_client.py. The only credential that the system currently requires is a Global Fishing Watch API token, but in the future I hope to add some sort of GPS token, for extended visualization functionality.

4.3 State Management

Most of the system is effectively stateless, but there are a few pieces of state that live outside the main execution environment (i.e., outside the container).

The main external state is the Global Fishing Watch dataset itself. This is not stored locally and is accessed on demand through the API. In this sense, the system relies on a remote data store managed by Global Fishing Watch, and all fishing activity data is fetched dynamically at runtime rather than persisted.

The only internal state in the current implementation is the history stored in the FishingAgent. Right now, this exists in memory and is lost when the process stops, so it is not truly externalized. However, in a more complete version of the system, this history would likely be stored in a persistent database such as a lightweight key-value store (e.g., Redis) or a document store (e.g., MongoDB). In that case, each run could be indexed by its query parameters (date range and region), allowing the system to retrieve past analyses or avoid recomputation.

Configuration state, such as API tokens and base URLs, is also stored outside the container in a config file. This keeps sensitive information separate from the codebase and allows the system to be deployed across different environments without modification.

Importantly, no raw API responses or intermediate datasets are persisted externally in the current design. Only summarized outputs (if persistence is added) would be stored, which helps reduce storage requirements and limits exposure to potentially noisy or unverified data.

4.4 Edge Consideration

This would be fairly easy to deploy on the edge, as much of the actual data storage is stored in the Global Fishing Watch API. However, if this were to deployed on an edge device, a smaller model than the Phi-3-mini-4k-instruct that the system currently uses should be chosen. The current model takes up a lot of memory, and further research would need to be conducted to find a model that has similar output correctness with a smaller footprint.

5 Failure Mode Analysis

Failure Mode	Type	Severity	Detection Method	Blast Radius	Mitigation	Security Risk?
Invalid API URL	Transient	Medium	API error / timeout	Data retrieval stage	Retry with backoff, validate endpoint	No
Invalid API Token	Permanent	High	Auth error response	Whole system	Fail fast, alert user	No
Edge-case Data / Degenerate Model Output	Reasoning	Medium	Weak/empty clusters, no anomalies	Analysis + LLM output	Add data validation, minimum thresholds	No

6 Failure Injection Report

6.1 Failure Injection One:

For the first failure injection, I decided to break the API endpoint. I changed the BASE_URL from the correct one to "https://bad-gateway.api.globalfishingwatch.org/v3", which isn't real. This should cause the API call to fail and error out, stopping the system in it's tracks.

```
requests.exceptions.SSLError: HTTPSConnectionPool(host='bad-gateway.api.globalfishingwatch.o
rg', port=443): Max retries exceeded with url: /v3/4wings/report?datasets%5B0%5D=public-glob
al-fishing-effort%3Alatest&date-range=2025-01-01%2C2025-01-15&spatial-resolution=LOW&tempora
l-resolution=DAILY&group-by=VESSEL_ID&format=JSON&region-id=5668&region-dataset=public-eez-a
reas (Caused by SSLError(SSLEOFError(8, '[SSL: UNEXPECTED_EOF_WHILE_READING] EOF occurred in
 violation of protocol (_ssl.c:1081)')))
(.venv) [magnet@nimona agentic-prototype]$
```

Figure 1: Results of first injected failure

This is a hard error where the system fails during data retreival and no downstream processing occurs.

The failure originates in GFWClient and propagates immediately to FishingAgent, but the pipeline halts before GeoFilter.

Implementing some sort of endpoint validation before the pipeline begins might help prevent or mitigate this error.

6.2 Failure Injection Two:

I decided to break detect_anomalies by marking everything as normal (gdf["anomaly"] = 1).

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
able higher rate limits and faster downloads.
No anomalies detected.
(.venv) [magnet@nimona agentic-prototype]$
```

Figure 2: Results of second injected failure

This is a silent failure where the output appears valid but is actually incorrect.

This failure originates in VesselAnalyzer, passes through FishingAgent, and then the LLMAgent reinforces it by explaining erroneous input.

To prevent this, putting a output validation guardrail to require a minimum anomaly rate or performing sanity checks on model outputs might be helpful.

7 Reflection

7.1 The PA2 Comparison

This pipeline has less boundaries than in PA2. There is less possibilty for error from the data input side. Switching from plan CSV to an API should help with the data filtering and limit issues. It is also easier to detect and prevent failure, as the API has error codes built in. Overall, there are less boundaries that are easier to enforce. Both systems have silent error possibilities without checks from the LLM. I will add that for PA4.

7.2 My Most Fragile Component

The most fragile component is the anomalies detection. If the system erroneously makes nothing as an anomaly, or everything as an anomaly, there are no checks in place to ensure sane output. A simple threshold sanity check should detect this failure in production before a user does.

7.3 The Model Replacement Test

I don't think anything would really change. The model I am currently using is very small, so odds are if the model gets changed the output might be even better. The LLM is to support the system, not be the end-all be-all.

7.4 What I Would Build Differently

I wish I had some way for the user to visualize the ships on a map. I think pointing out the anomalies versus non-anomalies would be helpful. I also wish I had put more checks and bounds in place, like have an API-output sanity check before the LLM reasoning output.

A requirements.txt

```
accelerate==1.13.0
annotated-doc==0.0.4
anyio==4.13.0
branca==0.8.2
certifi==2026.2.25
charset-normalizer==3.4.7
click==8.3.2
cuda-bindings==13.2.0
cuda-pathfinder==1.5.2
cuda-toolkit==13.0.2
filelock==3.25.2
folium==0.20.0
fsspec==2026.3.0
geopandas==1.1.3
h11==0.16.0
hf-xet==1.4.3
httpcore==1.0.9
httpx==0.28.1
huggingface_hub==1.10.1
idna==3.11
Jinja2==3.1.6
joblib==1.5.3
markdown-it-py==4.0.0
MarkupSafe==3.0.3
mdurl==0.1.2
mpmath==1.3.0
networkx==3.6.1
numpy==2.4.4
nvidia-cublas==13.1.0.3
nvidia-cuda-cupti==13.0.85
nvidia-cuda-nvrtc==13.0.88
nvidia-cuda-runtime==13.0.96
nvidia-cudnn-cu13==9.19.0.56
nvidia-cufft==12.0.0.61
nvidia-cufile==1.15.1.6
nvidia-curand==10.4.0.35
nvidia-cusolver==12.0.4.66
nvidia-cusparse==12.6.3.3
nvidia-cusparselt-cu13==0.8.0
nvidia-nccl-cu13==2.28.9
nvidia-nvjitlink==13.0.88
nvidia-nvshmem-cu13==3.4.5
nvidia-nvtx==13.0.85
packaging==26.0
pandas==3.0.2
```

```
psutil==7.2.2
Pygments==2.20.0
pyogrio==0.12.1
pyproj==3.7.2
python-dateutil==2.9.0.post0
PyYAML==6.0.3
regex==2026.4.4
requests==2.33.1
rich==14.3.3
safetensors==0.7.0
scikit-learn==1.8.0
scipy==1.17.1
setuptools==81.0.0
shapely==2.1.2
shellingham==1.5.4
six==1.17.0
sympy==1.14.0
threadpoolctl==3.6.0
tokenizers==0.22.2
torch==2.11.0
tqdm==4.67.3
transformers==5.5.3
triton==3.6.0
typer==0.24.1
typing_extensions==4.15.0
urllib3==2.6.3
xyzservices==2026.3.0
```